

TCP Chat Learning Handbook

Python sockets, one-thread-per-client concurrency, and a mastery tree

How to use this handbook

Read a concept, predict the behavior, implement one small step in Vim, test it, then record evidence in the knowledge tree and learning log. A checked box means you can explain and demonstrate the skill.

Lesson 1: TCP and Python sockets

Learning outcomes

After this lesson, you should be able to explain the client-server relationship, build a one-client TCP exchange, and diagnose common address and byte/string mistakes.

1. The mental model

A network application has two endpoints. The server owns a known address and waits. The client starts a connection to that address. Once connected, both sides possess a socket representing one endpoint of a full-duplex byte stream.

TCP provides:

- reliable, ordered delivery of bytes;
- duplicate suppression;
- flow and congestion control;
- a connection with a beginning and an end.

TCP does not provide messages. If one side calls `sendall()` twice, the other side might receive those bytes in one `recv()` call or several calls. Message boundaries must eventually be designed by your application protocol.

2. Addresses and ports

An IPv4 endpoint is commonly represented as `(host, port)`.

- `127.0.0.1` is loopback: only this computer can reach it.
- `0.0.0.0` is a server bind address meaning all local interfaces. It is not a

useful destination for a client.

- A port identifies the receiving application on a host.
- Ports below 1024 commonly require elevated privileges; use 5000 while

learning.

The server binds. The client connects. These are different roles.

3. Socket vocabulary

`socket.AF_INET` selects IPv4. `socket.SOCK_STREAM` selects a reliable byte stream, which normally means TCP.

Server lifecycle:

```
socket -> bind -> listen -> accept -> recv/send -> close
```

Client lifecycle:

```
socket -> connect -> send/recv -> close
```

`accept()` does not reuse the listening socket for conversation. It returns a new connected socket and the client's address. The listening socket remains available to accept future connections.

4. Bytes versus text

Sockets carry bytes. Python's `input()` returns text (`str`). Convert at the boundary:

```
outgoing_bytes = message.encode("utf-8")
incoming_text = data.decode("utf-8")
```

Keep data as bytes while transporting it and decode only when the application needs text. UTF-8 should be explicit when you first learn the conversion.

`sendall(data)` continues sending until all bytes have been handed to the operating system or an error occurs. `send(data)` may send only part of a large buffer.

`recv(1024)` means "receive up to 1024 bytes." It may return fewer. When it returns `b""`, the peer has closed its sending side and the receive loop should end.

5. Resource ownership

Sockets consume operating-system resources. A context manager closes a socket even when an exception occurs:

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    # use sock
    ...
```

The code that accepts a connected socket must decide who owns and closes it. Once worker threads are introduced, the worker should normally own that client socket.

6. Milestone A: one exchange

Implement this without threads:

1. Server creates, binds, and listens.
2. Client connects.
3. Client reads one line from the user and sends its UTF-8 bytes.
4. Server accepts once, receives once, decodes, and prints.
5. Both programs close cleanly.

Evidence of completion:

- You can run the server and client from separate terminals.
- You can explain why the socket passed to `recv()` is not the listening socket.
- You can explain why `decode()` belongs after `recv()`.

7. Experiments

Change one thing at a time and predict the result before running it:

1. Start the client while the server is stopped.
2. Run two clients when the server accepts only once.
3. Bind the server to a different port without changing the client.
4. Send an empty string.
5. Remove the server's `decode()` and inspect the printed value.

Record the exception names and explanations in the learning log.

8. Common failures

- `ConnectionRefusedError`: nothing is listening at the requested endpoint.
- `OSError: address already in use`: another process owns the port, or a recent

connection is still being cleaned up.

- Program appears frozen: blocking calls such as `accept()` and `recv()` are waiting; determine which event would allow progress.
- Garbled or failed decoding: sender and receiver disagree about the encoding, or a future protocol split a multi-byte character incorrectly.

Self-check

Answer without looking:

1. What does `listen()` change about a socket?
2. Why does `accept()` return another socket?
3. What exactly is guaranteed by TCP?
4. Is one `sendall()` guaranteed to equal one `recv()`?
5. What does `b""` from `recv()` mean?

Lesson 2: One thread per client

Learning outcomes

After this lesson, you should be able to separate connection acceptance from connection handling, start a worker with arguments, and reason about ownership, blocking, cleanup, and shared state.

1. Why the single-client server stops scaling

`accept()` blocks until a connection arrives. `recv()` blocks until bytes arrive or the connection ends. If the main server calls `recv()` for client A, it cannot return to `accept()` while A is idle. New clients wait unnecessarily.

The first concurrency design is:

```
main thread: accept -> start worker -> accept -> start worker -> ...
worker thread: recv from exactly one client until it disconnects
```

This model is easy to understand and suitable for a learning project. It is not the only model; later you can compare it with `asyncio` and thread pools.

2. Functions are values

A thread needs a callable to run later:

```
worker = threading.Thread(target=handle_client, args=(connection, address))
worker.start()
```

`target=handle_client` passes the function. `target=handle_client()` calls the function immediately in the current thread and passes its return value, which is a common beginner mistake.

3. Separation of responsibilities

The main thread should:

- own the listening socket;
- accept connections;
- create and start workers;
- eventually coordinate shutdown.

Each worker should:

- own one connected client socket;
- receive until that client disconnects;
- decode and process complete protocol messages;
- catch expected connection errors;
- remove the client from shared state later;
- close its socket exactly once.

This separation keeps the accept loop responsive.

4. The handler loop

Write a function with this shape, filling the body yourself:

```
def handle_client(connection, address):
    with connection:
        while True:
            # receive bytes
            # detect disconnect
            # decode and process data
            ...
```

The function must return when `recv()` yields `b""`. A loop that ignores this condition can spin forever, consuming CPU.

5. Starting threads

After every successful `accept()`:

1. Store both returned values using precise names.
2. Construct a `Thread` whose target is the handler.
3. Pass the connected socket and address through `args`.
4. Call `start()`, not `run()`.
5. Immediately let the main loop call `accept()` again.

Calling `run()` directly executes synchronously in the current thread. Calling `start()` asks Python to create the new thread, which then invokes your target.

6. Daemon versus non-daemon

Non-daemon threads keep the Python process alive until they finish. Daemon threads are stopped abruptly when only daemon threads remain. Abrupt stopping can skip cleanup, so begin with non-daemon threads and learn explicit shutdown later.

Pressing Ctrl-C raises `KeyboardInterrupt` in the main thread. A polished server will stop accepting, signal workers, close sockets, and join threads. That is a later milestone; first make connection handling correct.

7. Shared state and races

Broadcasting requires a shared collection of connected clients. Multiple threads may add, remove, or iterate over it. A sequence that looks like one idea in Python can consist of several operations that interleave with another thread.

A `threading.Lock` protects an invariant:

```
acquire lock -> inspect/change shared state -> release lock
```

Do not hold the lock while performing slow network sends if you can first take a safe snapshot. Otherwise one slow client can delay connection management.

The Global Interpreter Lock does not make your application-level invariants automatically safe. It is an implementation mechanism, not a replacement for reasoning about shared mutable state.

8. Milestone B: concurrent connections

Starting from the working single-client server:

1. Extract the connected-client code into `handle_client()`.
2. Put `accept()` inside an infinite loop.
3. Start a new thread after every acceptance.
4. Let each client send multiple messages before disconnecting.
5. Test with at least three terminals.

Evidence of completion:

- One idle client does not prevent another client from connecting and sending.
- Disconnecting one client does not stop the server.
- You can explain which thread owns each socket.
- You can explain the difference between `start()` and `run()`.

9. Debugging concurrency

Include the client address and current thread name in temporary debug messages. Reproduce bugs with a written sequence such as "A connects, B connects, A disconnects while B sends." Random repeated testing is less informative.

Useful questions:

- Which thread is blocked, and on what operation?
- Which object is shared?
- Who owns and closes this socket?
- Can another thread mutate this collection during iteration?
- Does every exit path perform cleanup?

Self-check

1. Why must the main thread avoid handling the whole client conversation?
2. What is wrong with `target=handle_client()`?
3. Who should close an accepted client socket?
4. Why can a shared list of clients need a lock?
5. Why should a broadcast avoid holding a lock during slow I/O?

TCP Chat Knowledge Tree

Updated: 2026-09-22

How to use this tree

Check a box only after producing the stated evidence. Use [~] for in progress, [x] for demonstrated, and [] for not started. Add links to commits, terminal transcripts, tests, or learning-log entries where useful.

0. Tooling foundations

- [~] Terminal workflow
 - [x] Run Python files from the shell
 - [] Manage two or more terminal sessions for server/client testing
 - [] Explain exit status and read a traceback from bottom to top
- [~] Vim workflow
 - [x] Open, edit, save, and quit a file
 - [] Navigate and edit comfortably without arrow-key dependence
 - [] Search, replace, undo, copy, paste, and use splits
- [] Git workflow
 - [] Initialize repository and make an intentional first commit
 - [] Inspect diffs before committing
 - [] Use small milestone commits with meaningful messages

1. Python foundations

- [~] Functions and control flow
 - [x] Define and call a function
 - [] Explain `if __name__ == "__main__"`
 - [] Use loops with clear termination conditions
- [~] Data representation
 - [] Explain `str` versus `bytes`
 - [] Encode and decode UTF-8 at system boundaries
 - [] Explain tuples and argument unpacking
- [] Resource management
 - [] Use `with` to guarantee socket cleanup
 - [] Explain exception propagation and `try/finally`

2. Networking foundations

- [~] TCP mental model
 - [] Explain client, server, host, port, and endpoint
 - [] Explain what TCP guarantees and what it does not
 - [] Explain why TCP is a byte stream without message boundaries
- [~] Socket lifecycle
 - [] Implement `socket -> bind -> listen -> accept`
 - [] Implement `socket -> connect`
 - [] Send with `sendall()` and receive with `recv()`

- ☐ Detect orderly disconnect through `b""`
- ☐ Failure diagnosis
- ☐ Reproduce and explain `ConnectionRefusedError`
- ☐ Reproduce and explain address/port mismatch
- ☐ Identify which blocking call is waiting

3. Concurrency foundations

- ☐ Thread model
- ☐ Explain process versus thread
- ☐ Pass a callable and arguments to `Thread`
- ☐ Explain `start()` versus `run()`
- ☐ Thread-per-client server
- ☐ Keep acceptance in the main thread
- ☐ Handle each connected socket in one worker
- ☐ Demonstrate three simultaneous connections
- ☐ Disconnect one client without affecting others
- ☐ Shared-state safety
- ☐ Identify the shared client collection
- ☐ State the invariant protected by a lock
- ☐ Broadcast from a safe snapshot
- ☐ Explain one possible race condition

4. Chat protocol

- ☐ Framing
- ☐ Demonstrate that `sendall()` and `recv()` boundaries can differ
- ☐ Implement newline-delimited or length-prefixed messages
- ☐ Buffer incomplete frames correctly
- ☐ Message semantics
- ☐ Design join, chat, leave, and error messages
- ☐ Validate malformed input without crashing the server
- ☐ Document encoding and maximum message size
- ☐ Identity
- ☐ Accept and validate usernames
- ☐ Handle duplicate usernames deliberately

5. Reliability and testing

- ☐ Error handling and cleanup
- ☐ Handle abrupt client disconnects
- ☐ Remove dead sockets from shared state
- ☐ Shut down server and workers deliberately

- [] Automated tests
 - [] Unit-test protocol encoding and parsing
 - [] Integration-test server/client communication
 - [] Test two concurrent clients without manual typing
- [] Observability
 - [] Replace temporary prints with structured logging
 - [] Include useful connection context without leaking message contents

6. CLI and packaging

- [] Command-line interface
 - [] Parse server host and port
 - [] Parse client host, port, and username
 - [] Provide helpful `--help` output and validation
- [] Package structure
 - [] Move code into `src/tcp_chat/`
 - [] Create `pyproject.toml`
 - [] Expose a `tcp-chat` console command
- [] Installation and release
 - [] Install locally in an isolated environment
 - [] Build wheel and source distribution
 - [] Install and smoke-test the built wheel

7. Stretch branches

- [] Compare thread-per-client with `asyncio`
- [] Add chat rooms and private messages
- [] Add TLS and explain authentication versus encryption
- [] Measure behavior with many idle connections

Current focus

Milestone A: implement and explain a one-client TCP exchange. Do not begin the threaded implementation until the three evidence items in Lesson 1 are met.